

Enhanced Entity-Relationship Modeling

Lecture 08

Borough of Manhattan Community College

Chapter Objectives

In this lecture you will learn:

- The limitations of the basic concepts of the Entity-Relationship (**ER**) model and the requirements to represent more complex applications using additional data modeling concepts.
- The most useful additional data modeling concepts of the Enhanced Entity-Relationship (**EER**) model.
- A diagrammatic technique for displaying these additional data modeling concepts in an **EER** diagram using the Unified Modeling Language (**UML**).

Superclasses and Subclasses

- ❑ An entity type represents a set of entities of the same type such as **Staff**, **Branch**, and **PropertyForRent**.

- ❑ We can also form entity types into a hierarchy containing *superclasses* and *subclasses*:
 - **Superclass**: An entity type that includes subgroupings of its occurrences, which must be represented in a data model.

 - **Subclass**: A distinct subgrouping of occurrences of an entity type, which must be represented in a data model.

Superclasses and Subclasses

- ❑ Entity types that have distinct *subclasses* are called *superclasses*.

For example, the entities that are members of the **Staff** entity type may be classified as **Manager**, **SalesPersonnel**, and **Secretary**.

- ❑ In other words, the **Staff** entity is referred to as the *superclass* of the **Manager**, **SalesPersonnel**, and **Secretary** *subclasses*.

- ❑ Some *superclasses* may contain overlapping *subclasses*.

For example a member of staff who is both a Manager and a member of Sales Personnel.

In this example, **Manager** and **SalesPersonnel** are overlapping *subclasses* of the **Staff** *superclass*.

Superclasses and Subclasses

- ❑ Each member of a *subclass* is also a member of the *superclass*.
In other words, the entity in the *subclass* is the same entity in the *superclass*, but has a distinct role.
- ❑ On the other hand, not every member of a *superclass* is necessarily a member of a *subclass*; for example, members of staff without a distinct job role such as a Manager or a member of Sales Personnel.
- ❑ If all staff attributes and those specific to particular jobs are described by a single **Staff** entity, this may result in a lot of *nulls* for the job-specific attributes.

Superclasses and Subclasses

- ❑ To illustrate these points, consider the relation called **AllStaff** which holds the details of all members of staff, no matter what position they hold.

staffNo	name	position	salary	mgrStartDate	bonus	sales Area	car Allowance	typing Speed
SL21	John White	Manager	30000	01/02/95	2000			
SG37	Ann Beech	Assistant	12000					
SG66	Mary Martinez	Sales Manager	27000			SA1A	5000	
SA9	Mary Howe	Assistant	9000					
SL89	Stuart Stern	Secretary	8500					100
SL31	Robert Chin	Snr Sales Asst	17000			SA2B	3700	
SG5	Susan Brand	Manager	24000	01/06/91	2350			

Attribute Inheritance

- ❑ An entity in a *subclass* represents the same "real world" object as in the *superclass*, and may possess *subclass-specific* attributes, as well as those associated with the *superclass*.
- ❑ For example, a member of the **SalesPersonnel** *subclass* inherits all the attributes of the **Staff** *superclass*, such as *staffNo*, *name*, *position*, and *salary* together with those specifically associated with the **SalesPersonnel** *subclass*, such as *salesArea* and *carAllowance*.
- ❑ A *subclass* is an entity in its own right and so it may also have one or more *subclasses*.
- ❑ An entity and its *subclasses* and their *subclasses*, and so on, is called a *type hierarchy*.

Attribute Inheritance

- ❑ Type hierarchies are known by a variety of names, including:
 - ***Specialization hierarchy*** (**Manager** is a specialization of **Staff**).
 - ***Generalization hierarchy*** (**Staff** is a generalization of **Manager**).

- ❑ A *subclass* with more than one *superclass* is called a ***shared subclass***. A member of a shared *subclass* must be a member of the associated *superclasses*.

- ❑ As a consequence, the attributes of the *superclasses* are inherited by the shared *subclass*, which may also have its own additional attributes. This process is referred to as ***multiple inheritance***.

Specialization Process

- ❑ **Specialization:** The process of maximizing the differences between members of an entity by identifying their distinguishing characteristics.
- ❑ Specialization is a top-down approach to defining a set of *superclasses* and their related *subclasses*.
- ❑ The set of *subclasses* is defined on the basis of some distinguishing characteristics of the entities in the *superclass*.
- ❑ When we identify a set of *subclasses* of an entity type, we then associate attributes specific to each *subclass* (where necessary), and also identify any relationships between each *subclass* and other entity types or *subclasses* (where necessary).

Specialization Process (Example)

- ❑ Consider a model where all members of staff are represented as an entity called **Staff**.
- ❑ If we apply the process of specialization on the **Staff** entity, we attempt to identify differences between members of this entity, such as members with distinctive attributes and/or relationships.
- ❑ As described earlier, staff with the job roles of Manager, Sales Personnel, and Secretary have distinctive attributes and therefore we identify **Manager**, **SalesPersonnel**, and **Secretary** as *subclasses* of the **Staff** *superclass*.

Generalization Process

- ❑ **Generalization:** The process of minimizing the differences between entities by identifying their common characteristics.
- ❑ The process of generalization is a bottom-up approach, that results in the identification of a generalized *superclass* from the original entity types.
- ❑ As the process of generalization can be viewed as the reverse of the specialization process, we refer to this modeling concept as Specialization/Generalization.

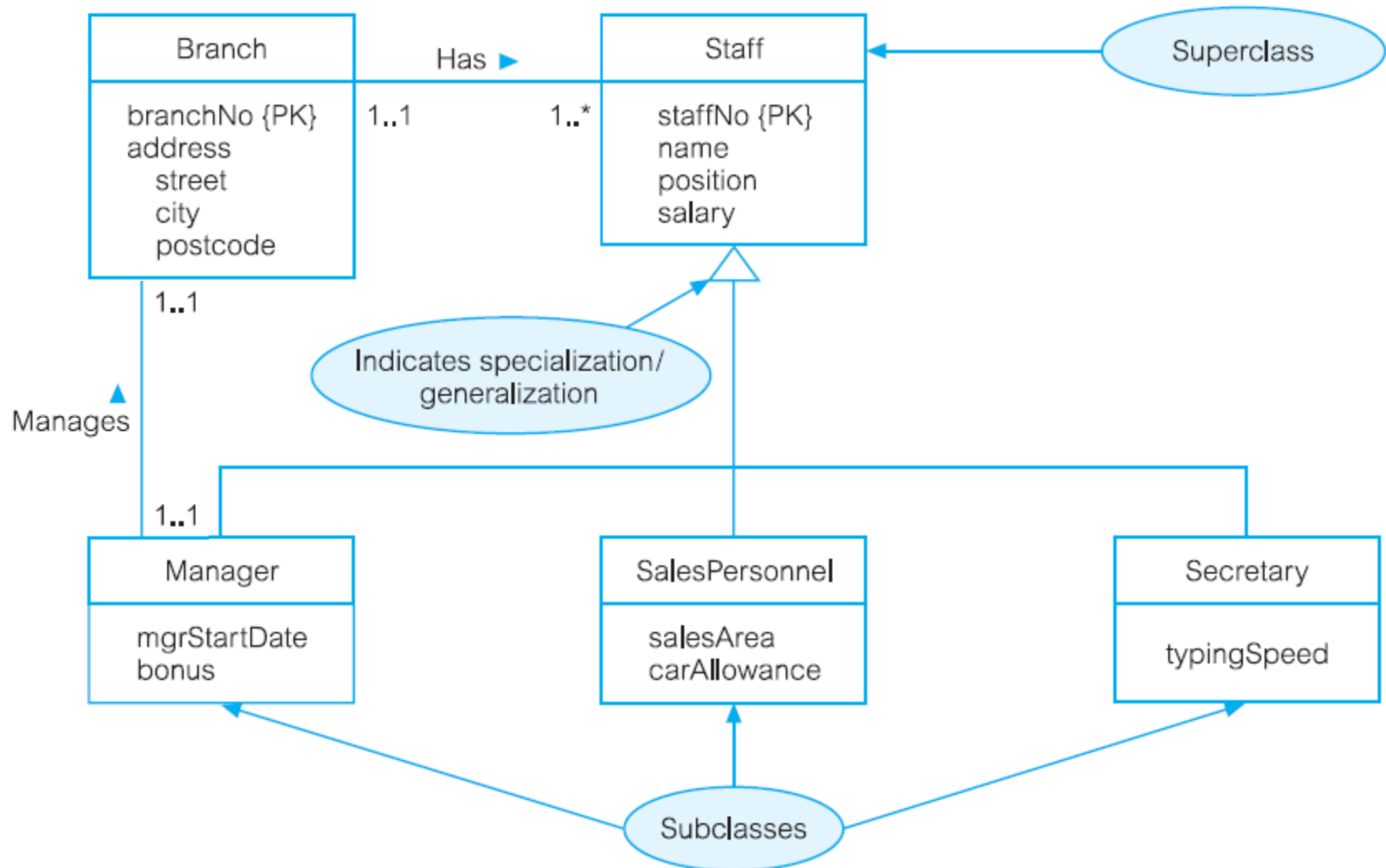
Generalization Process (Example)

- ❑ Consider a model where **Manager**, **SalesPersonnel**, and **Secretary** are represented as distinct entity types.
- ❑ If we apply the process of generalization on these entities, we attempt to identify similarities between them, such as common attributes and relationships.
- ❑ As stated earlier, these entities share attributes common to all staff, and therefore we identify **Manager**, **SalesPersonnel**, and **Secretary** as *subclasses* of a generalized **Staff** *superclass*.

Representation of Specialization/Generalization

- ❑ UML has a special notation for representing inheritance hierarchies. For example, the **Staff** *superclass* and the **Manager**, **SalesPersonnel**, and **Secretary** *subclasses* can be represented in an **EER** diagram, as follows:
 - The **Staff** *superclass* and the *subclasses* are represented as rectangles.
 - Attributes that are common to all *subclasses* are listed in the lower section of the rectangle representing the *superclass*.
 - Attributes that are specific to a given *subclass* are listed in the lower section of the rectangle representing that *subclass*.
 - The *subclasses* are attached by lines to a triangle that points toward the *superclass*.

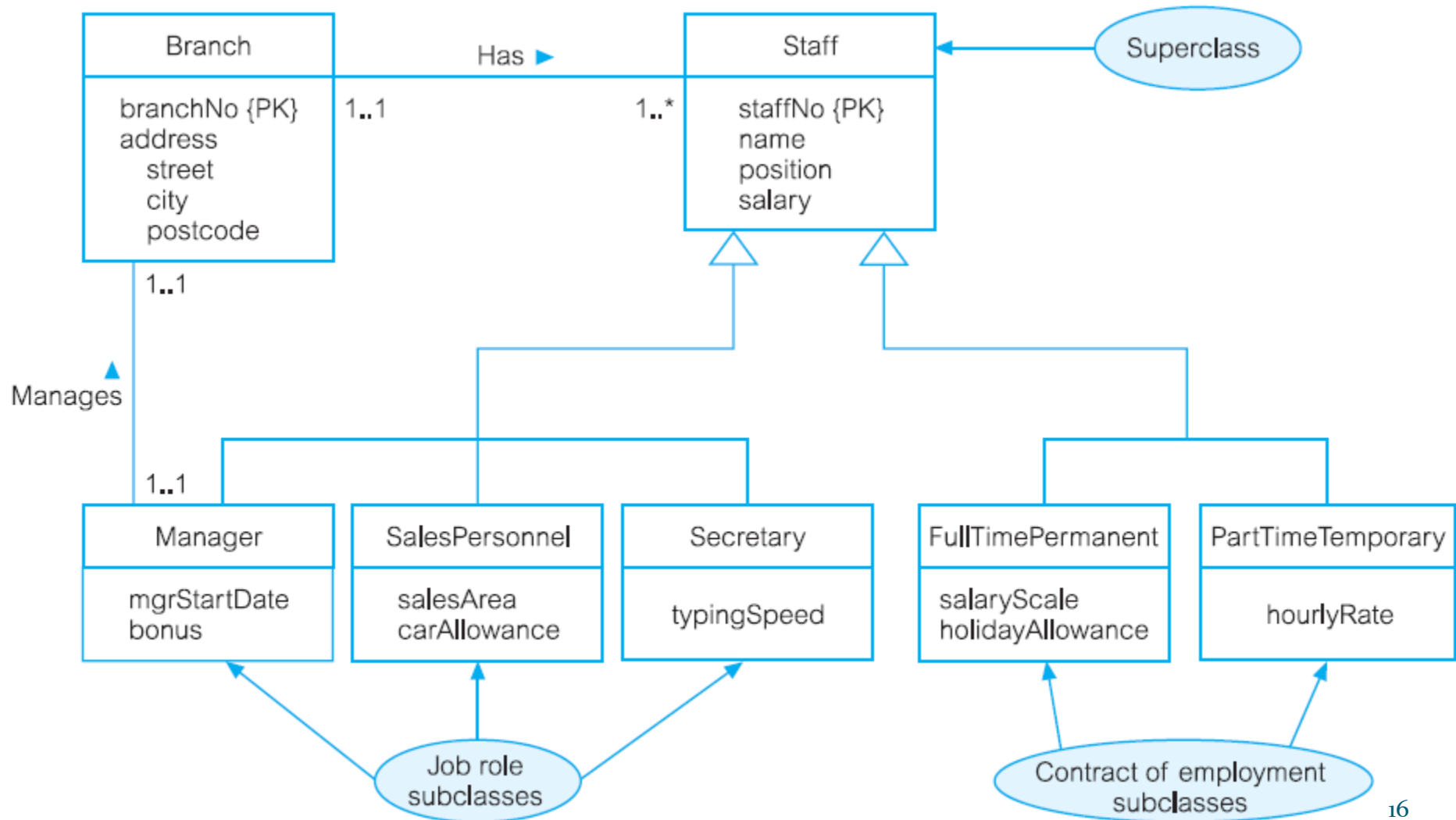
Representation of Specialization/Generalization



Representation of Specialization/Generalization

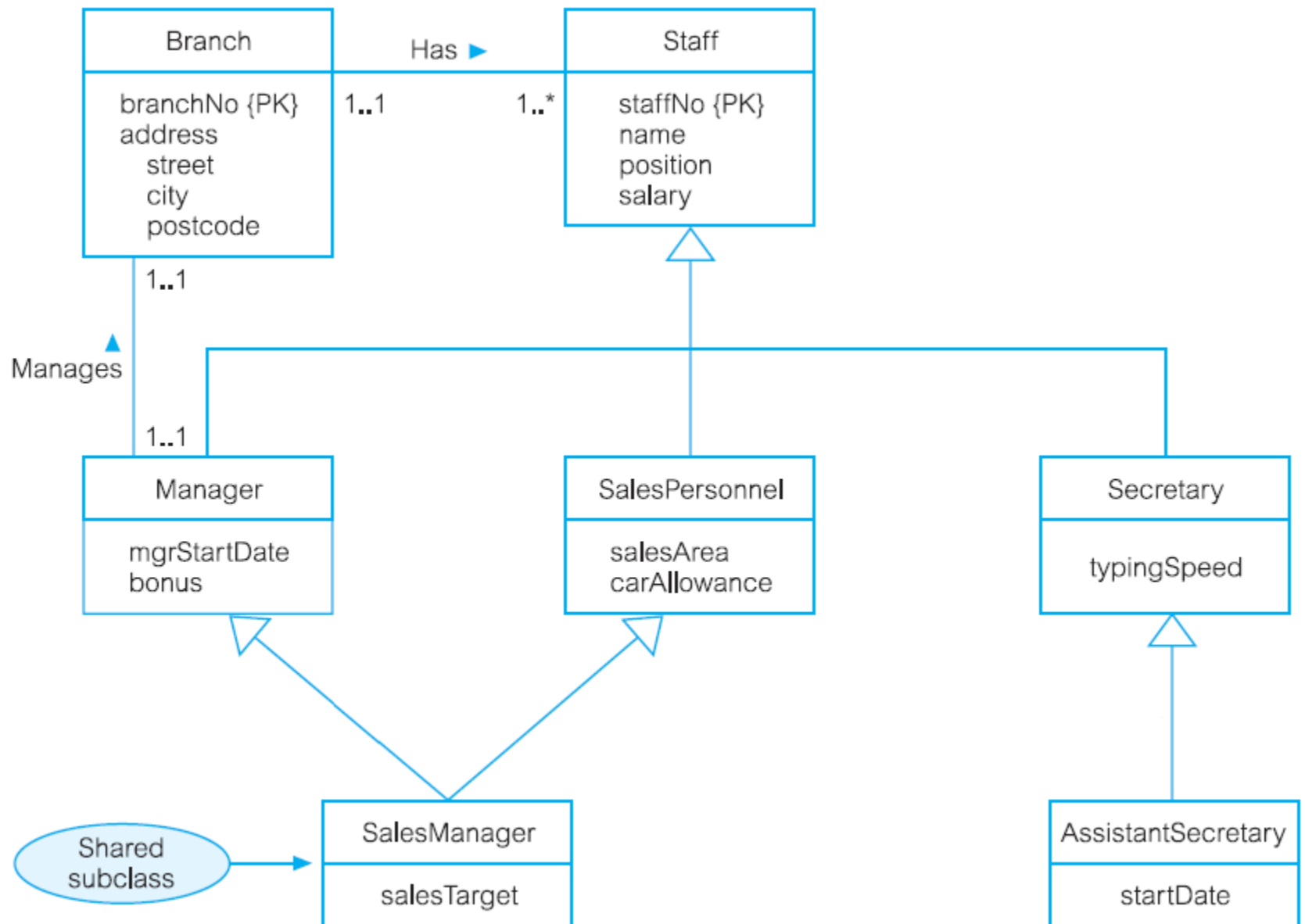
- ❑ We may have several specializations of the same entity based on different distinguishing characteristics.
- ❑ For example, another specialization of the **Staff** entity may produce the *subclasses* **FullTimePermanent** and **PartTimeTemporary**, which distinguishes between the types of employment contract for members of staff.
- ❑ The attributes that are specific to the **FullTimePermanent** *subclass* are *salaryScale* and *holidayAllowance*, and the attribute that is specific to the **PartTimeTemporary** subclass is *hourlyRate*.

Representation of Specialization/Generalization



Representation of Specialization/Generalization

- ❑ The *subclass* **SalesManager** illustrates an example of a type hierarchy where the job roles are expanded to show a shared *subclass*. A member of **SalesManager** must be a member of the **SalesPersonnel** and **Manager** *subclasses* as well as the **Staff** *superclass*.
- ❑ As a consequence, the attributes of the **Staff** *superclass*, and the attributes of the *subclasses* **SalesPersonnel** and **Manager** are inherited by the **SalesManager** *subclass*, which also has its own additional attribute called *salesTarget*.
- ❑ Another example of a type hierarchy is the *subclass* **Secretary** with its own *subclass* called **AssistantSecretary**.



Participation Constraints

- ❑ The relationship between a *superclass* and any one of its *subclasses* is called a *superclass/subclass* relationship.
For example, **Staff/Manager** is a *superclass/subclass* relationship.
- ❑ **Participation constraint:** Determines whether every member in the *superclass* must participate as a member of a *subclass*.
- ❑ A participation constraint may be *mandatory* or *optional*.

Participation Constraints

- ❑ A superclass/subclass relationship with ***mandatory participation*** specifies that every member in the *superclass* must also be a member of a *subclass*.
- ❑ For example, the contract of employment is mandatory participation, which means that every member of staff must have a contract of employment.
- ❑ To represent *mandatory participation*, "**Mandatory**" is placed in curly brackets below the triangle that points towards the *superclass*.

Participation Constraints

- ❑ A superclass/subclass relationship with ***optional participation*** specifies that a member of a *superclass* need not belong to any of its *subclasses*.
- ❑ For example, the job role has optional participation, which means that a member of staff need not have an additional job role such as a **Manager**, **SalesPersonnel**, or **Secretary**.
- ❑ To represent *optional participation*, "**Optional**" is placed in curly brackets below the triangle that points towards the *superclass*.

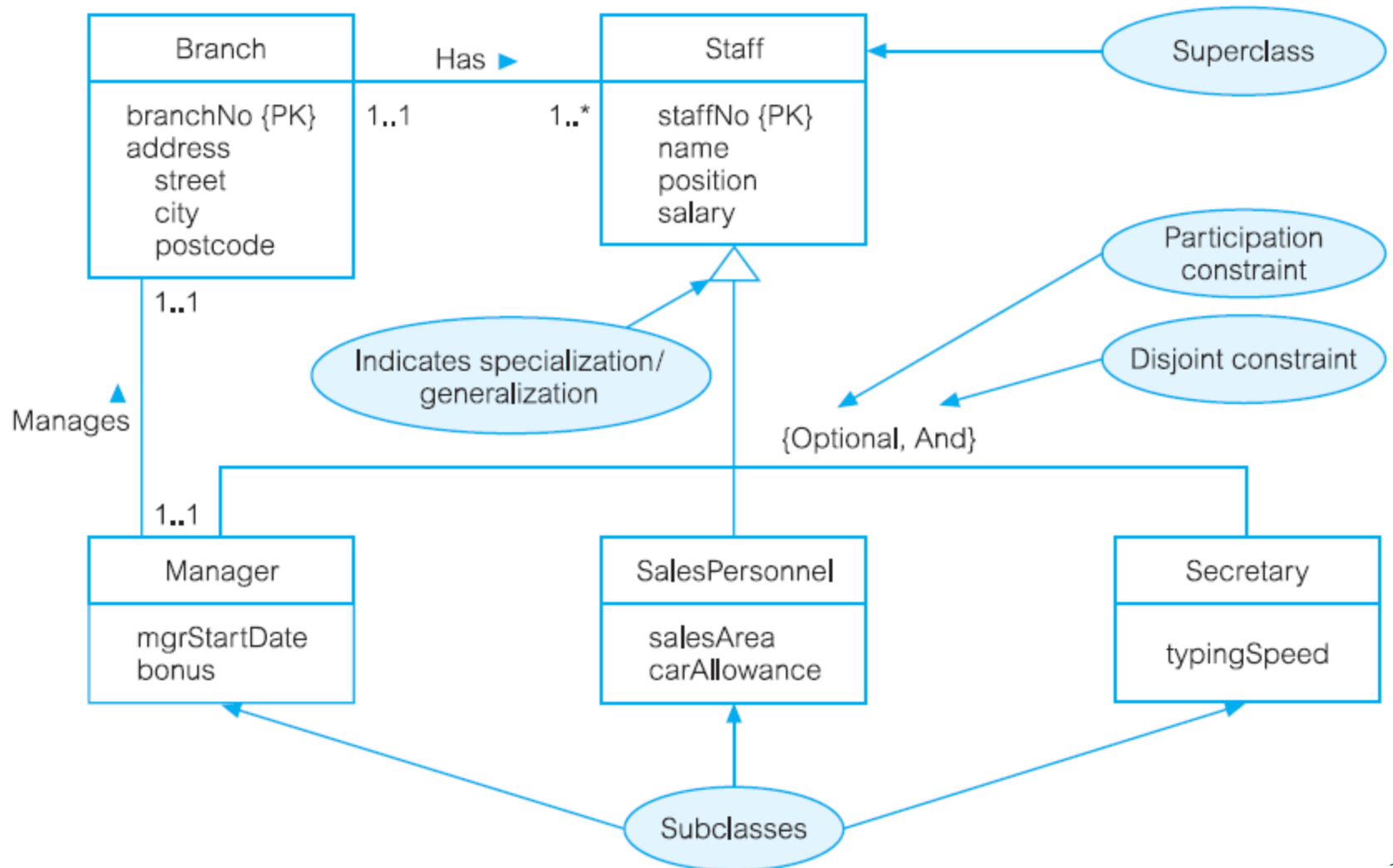
Disjoint Constraints

- ❑ ***Disjoint constraint***: Describes the relationship between members of the *subclasses* and indicates whether it is possible for a member of a *superclass* to be a member of one, or more than one, *subclass*.
- ❑ If the *subclasses* are *disjoint*, then an entity occurrence can be a member of only one of the *subclasses*.
- ❑ For example, the *subclasses* of the contract of employment are *disjoint*, which means that a member of staff must have a full-time permanent or a part-time temporary contract, but not both.
- ❑ To represent a *disjoint superclass/subclass* relationship, "Or" is placed next to the *participation constraint* within the curly brackets.

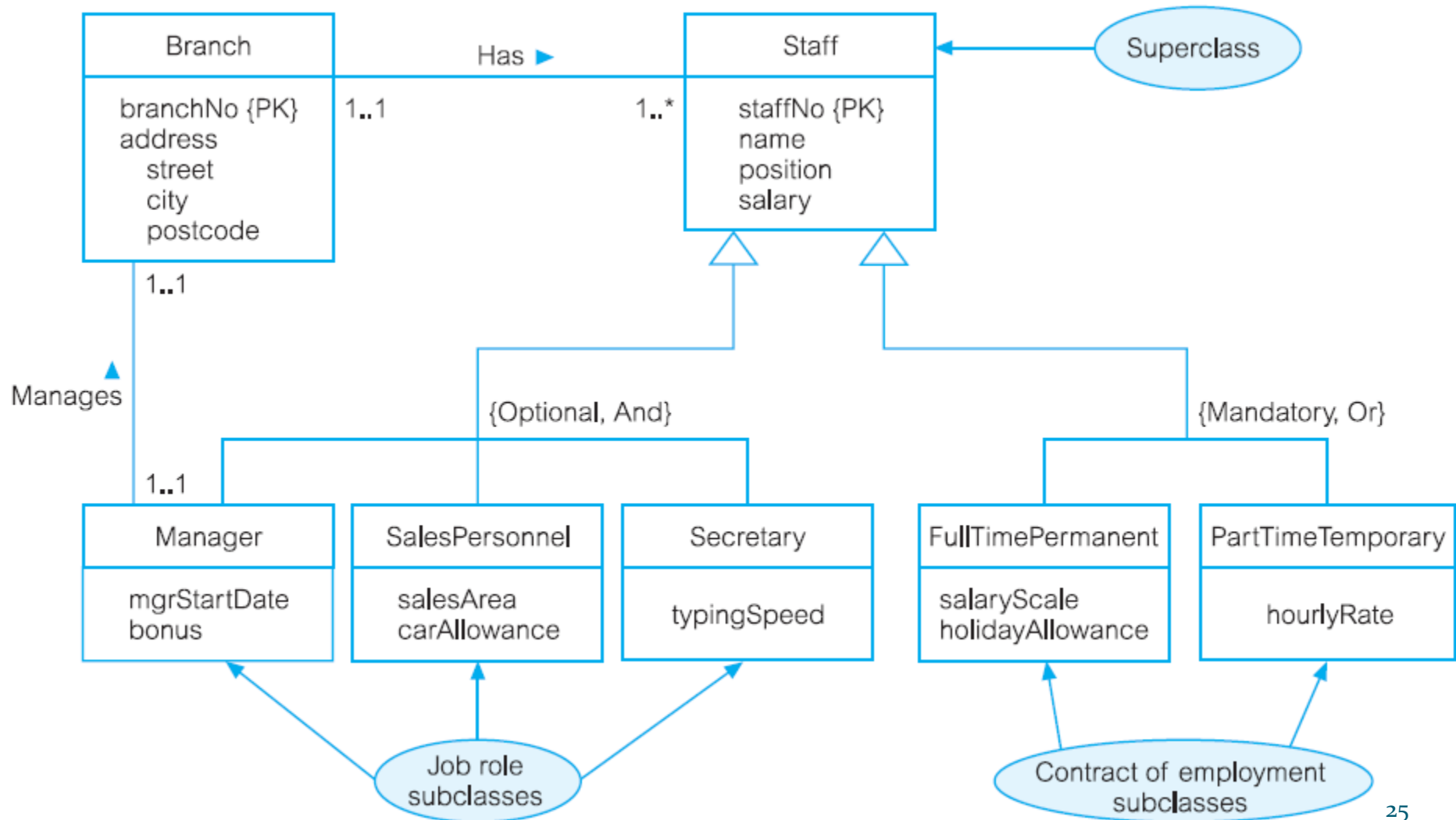
Disjoint Constraints

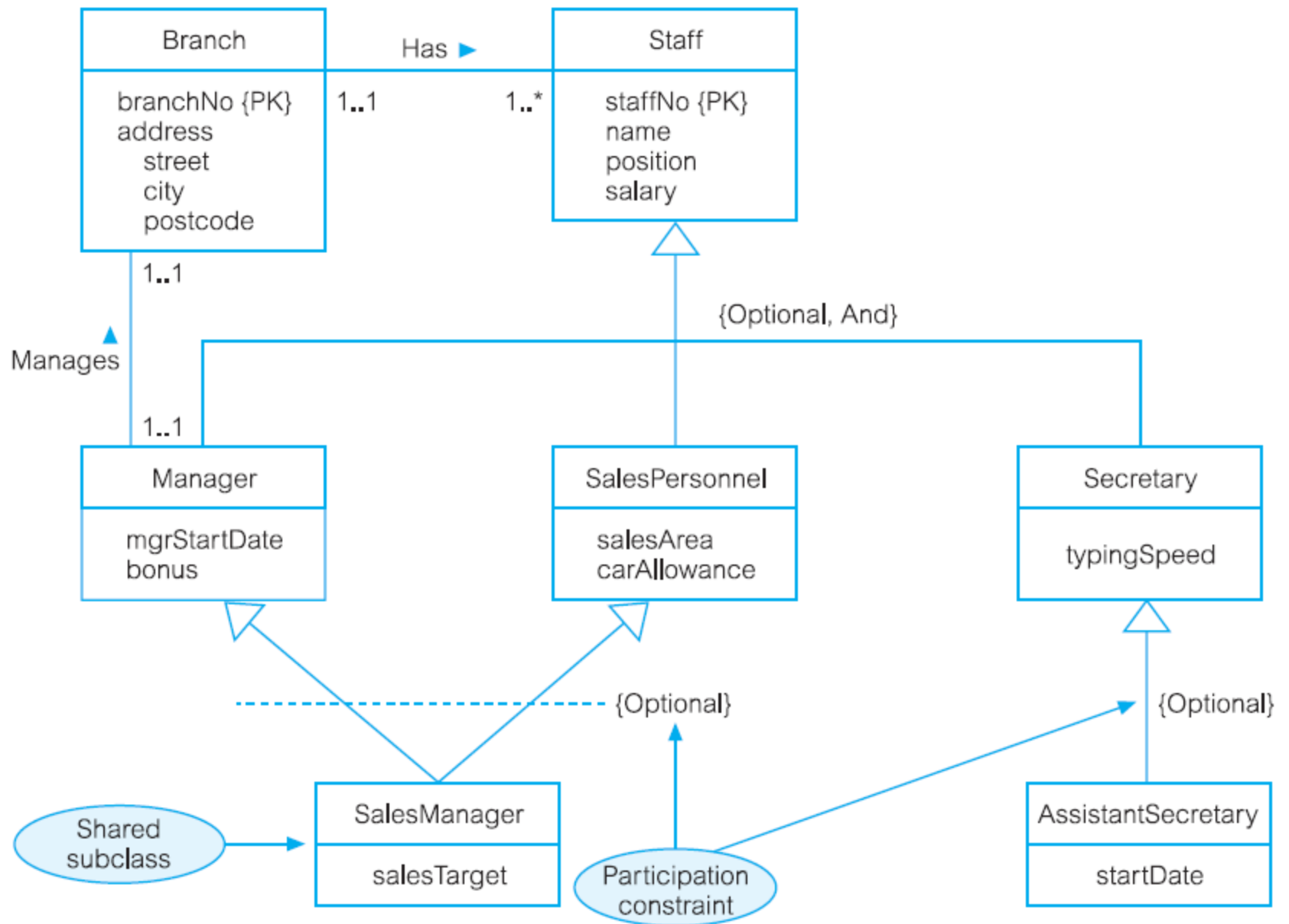
- ❑ If *subclasses* are not *disjoint* (called ***nondisjoint***), then an entity occurrence may be a member of more than one *subclass*.
- ❑ For example, the job role is *nondisjoint* since some entity occurrences can be members of both **Manager** and **SalesPersonnel** *subclasses*.
- ❑ To represent a *nondisjoint superclass/subclass* relationship, "**And**" is placed next to the participation constraint within the curly brackets.
- ❑ Note that it is not necessary to include the *disjoint constraint* for hierarchies that have a single *subclass* at a given level.
For this reason only the *participation constraint* is shown for the **SalesManager** and **AssistantSecretary** *subclasses*.

Representation of Specialization/Generalization



Representation of Specialization/Generalization



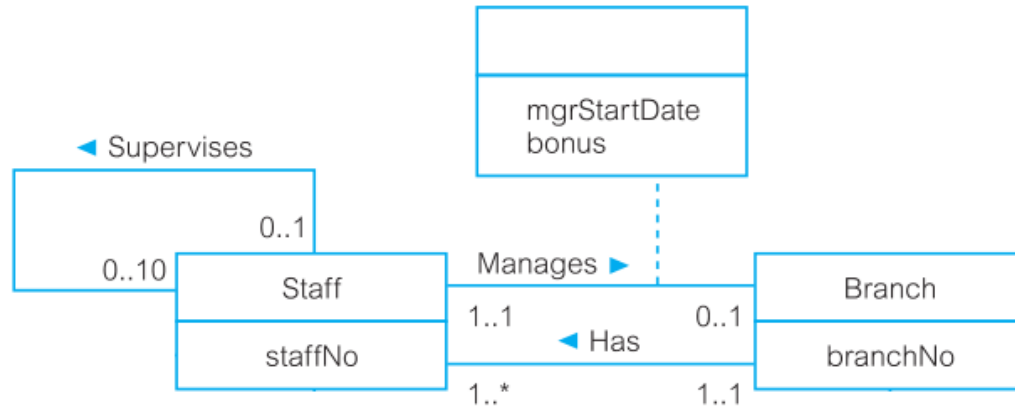


DreamHome Case Study

- ❑ The choice to use a database design methodology is dependent on the complexity of the enterprise being modeled and whether using the additional concepts of the **EER** model will help the process of database design.
- ❑ In **Lecture 07** we described the basic concepts necessary to build an **ER** model to represent the *DreamHome* case study.
- ❑ In this section, we show how specialization/generalization may be used to convert that **ER** model into an **EER** model.

Staff Supreclass

- Consider the **Staff** entity in the *ER* model. There are two key job roles, namely **Manager** and **Supervisor**.

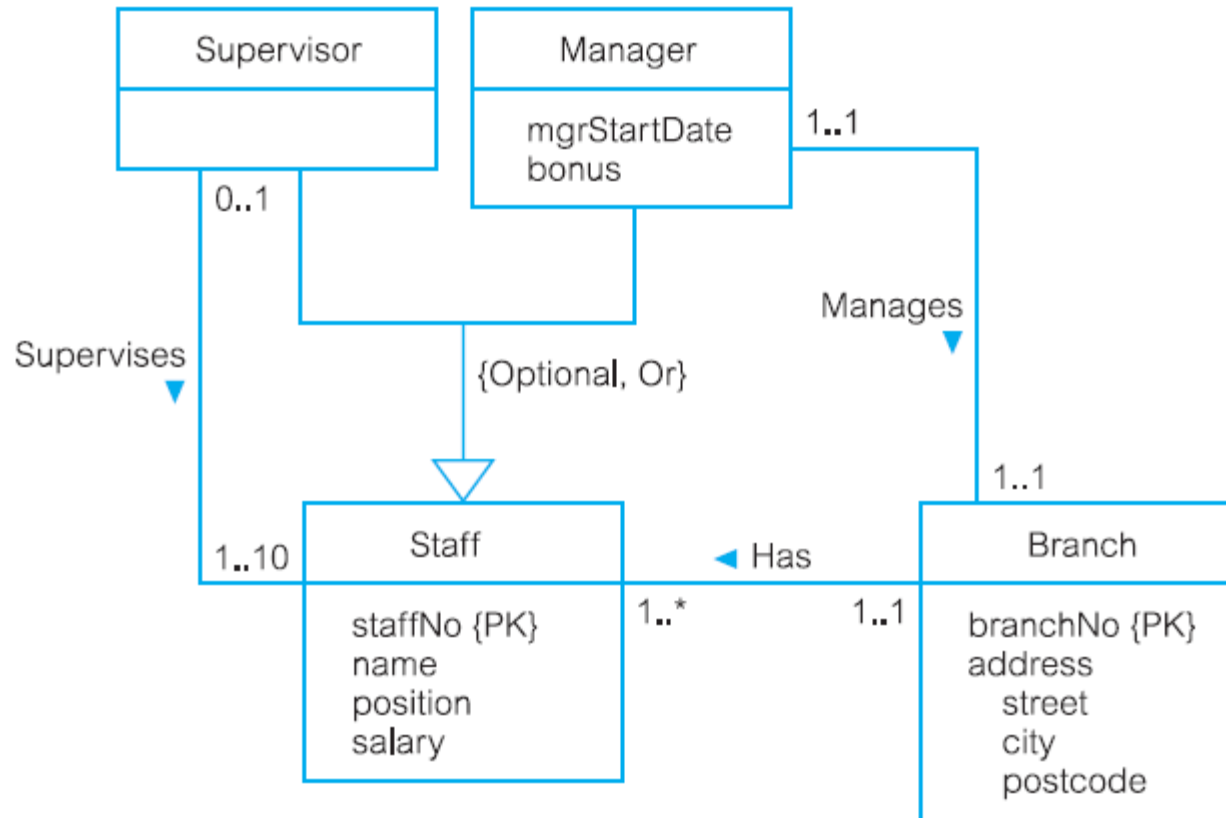


- We have three options as to how we may best model members of staff.
 - The first option is to represent all members of staff as a generalized **Staff** entity (as in the *ER* model),
 - the second option is to create three distinct entities **Staff**, **Manager**, and **Supervisor**,
 - and the third option is to represent the **Manager** and **Supervisor** entities as *subclasses* of a **Staff superclass**.

Staff *Supreclass*

- ❑ The option we select is based on the common attributes and relationships associated with each entity.
 - All attributes of the **Staff** entity are represented in the **Manager** and **Supervisor** entities.
 - The **Supervisor** entity does not have any additional attributes representing this job role. On the other hand, the **Manager** entity has two additional attributes: *mgrStartDate* and *bonus*.
 - Both the **Manager** and **Supervisor** entities are associated with distinct relationships, namely “*Manager Manages Branch*” and “*Supervisor Supervises Staff*”.
- ❑ Based on this information, we select the third option and create **Manager** and **Supervisor subclasses** of the **Staff superclass**.

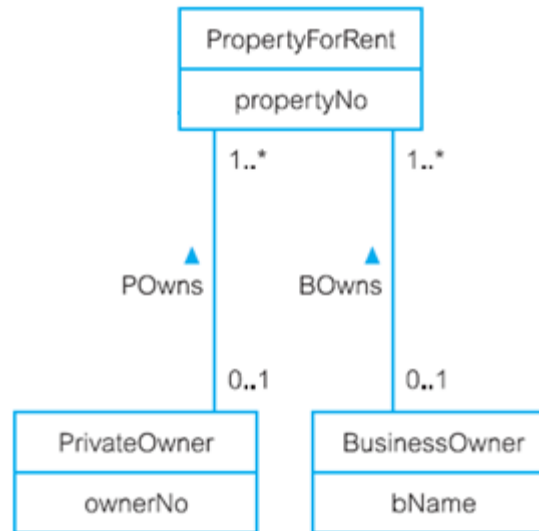
Staff Supreclass



- ❑ The specialization/generalization of the **Staff** entity is *optional* and *disjoint*, as not all members of staff are Managers or Supervisors, and a single member of staff cannot be both a Manager and a Supervisor.

Owner Supreclass

- ❑ Consider the relationship between owners and properties for rent. There are two types of owner (**PrivateOwner** and **BusinessOwner**).

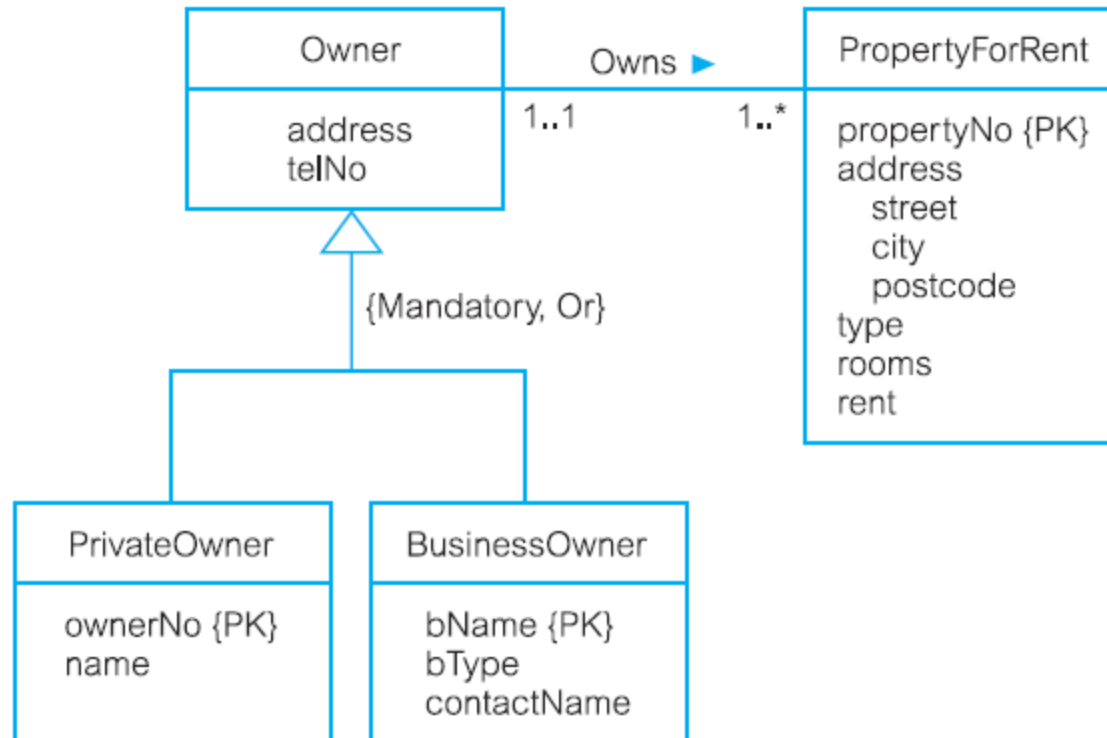


- ❑ We have three options as to how we may best model owners of property:
 1. The first option is to leave **PrivateOwner** and **BusinessOwner** as two distinct entities (as shown in the **ER** model),
 2. the second option is to represent both types of owner as a generalized **Owner** entity,
 3. and the third option is to represent the **PrivateOwner** and **BusinessOwner** entities as *subclasses* of an **Owner** *superclass*.

Owner *Supreclass*

- ❑ Before we are able to reach a decision we first examine the attributes and relationships associated with these entities.
 - **PrivateOwner** and **BusinessOwner** entities share common attributes, namely *address* and *telNo*.
 - Both types of owner have different attributes: *PrivateOwner* has distinct attributes *ownerNo* and *name*, and **BusinessOwner** has distinct attributes *bName*, *bType*, and *contactName*.
 - Both types of owner have a similar relationship with property for rent (namely “*PrivateOwner POwns PropertyForRent*” and “*BusinessOwner BOwns PropertyForRent*”).
- ❑ In this case, we create a *superclass* called **Owner**, with **PrivateOwner** and **BusinessOwner** as *subclasses*.

Owner Supreclass



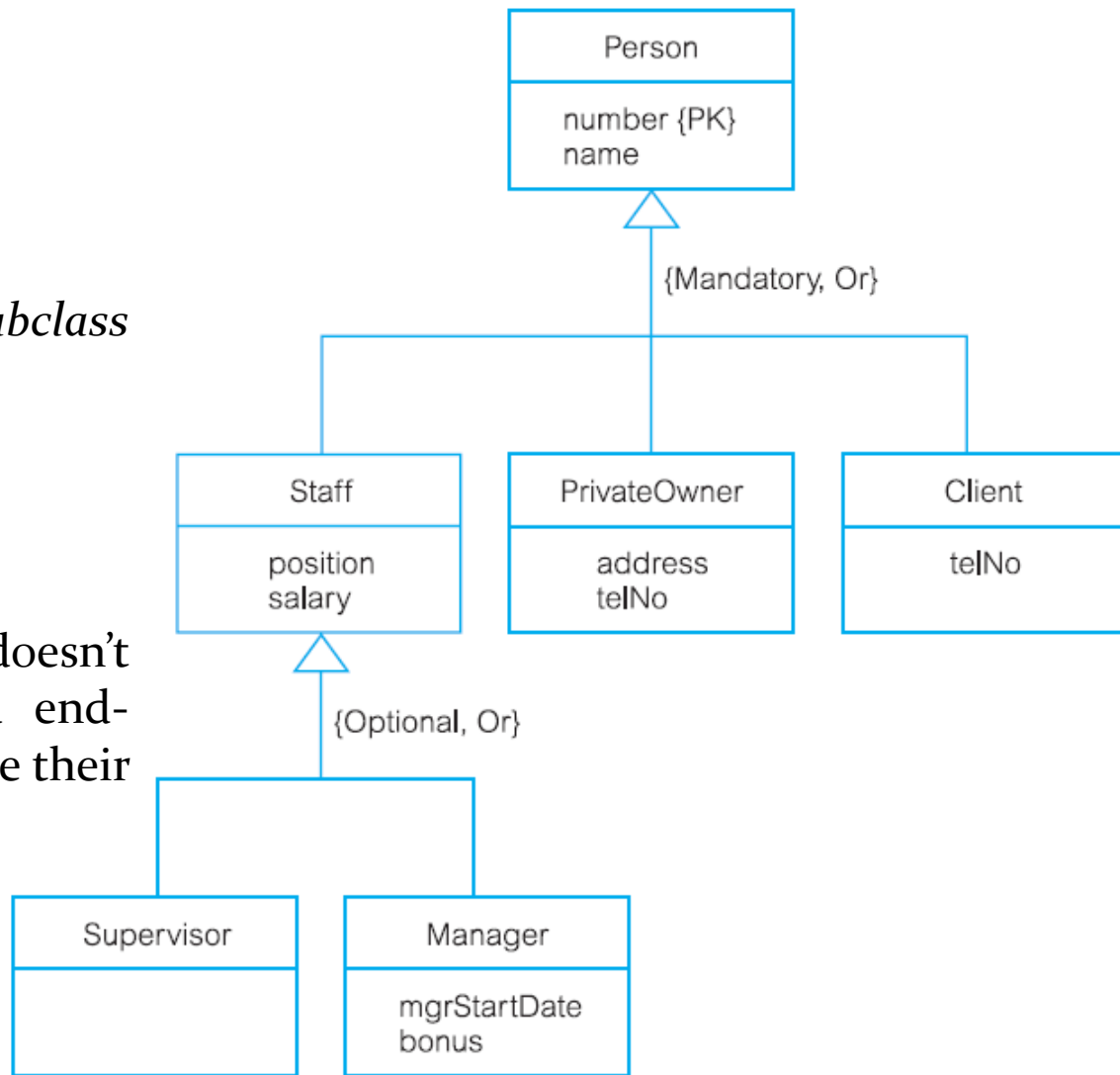
- ❑ The specialization/generalization of the Owner entity is *mandatory* and *disjoint*, as an owner must be either a private owner or a business owner, but cannot be both.

Person *Supreclass*

- ❑ There are several persons with common characteristics described in the data requirements specification of the *DreamHome* case study.
- ❑ For example, members of staff, private property owners, and clients all have ***number*** and ***name*** attributes.
- ❑ We could create a **Person** *superclass* with **Staff**, **PrivateOwner**, and **Client** as *subclasses*.
Note: the **Staff** *subclass* has its own *subclasses* (**Manager** and **Supervisor**).

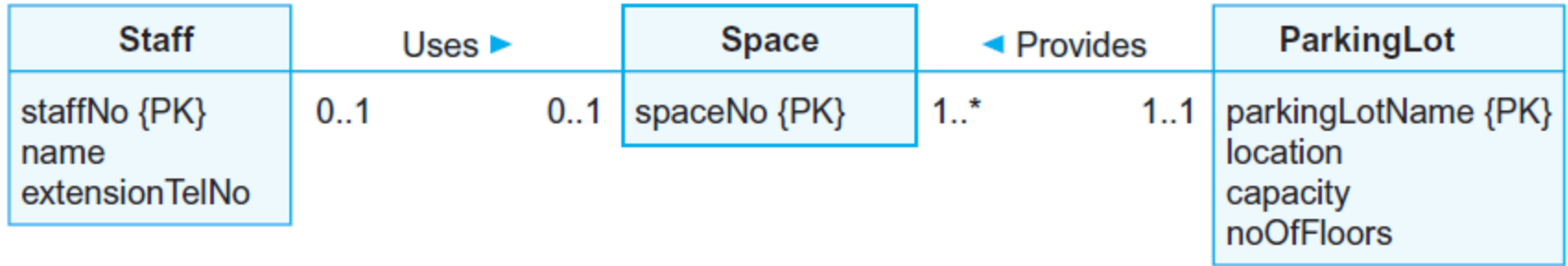
Person Supreclass

- **PrivateOwner** is already a *subclass* of the **Owner** superclass.
- Adding the **Person** superclass doesn't allow database designers and end-users to accurately communicate their understanding of the data.



- ❑ Based on this information, we choose to leave **Staff**, **PrivateOwner**, and **Client** as distinct entities.

Exercise 1

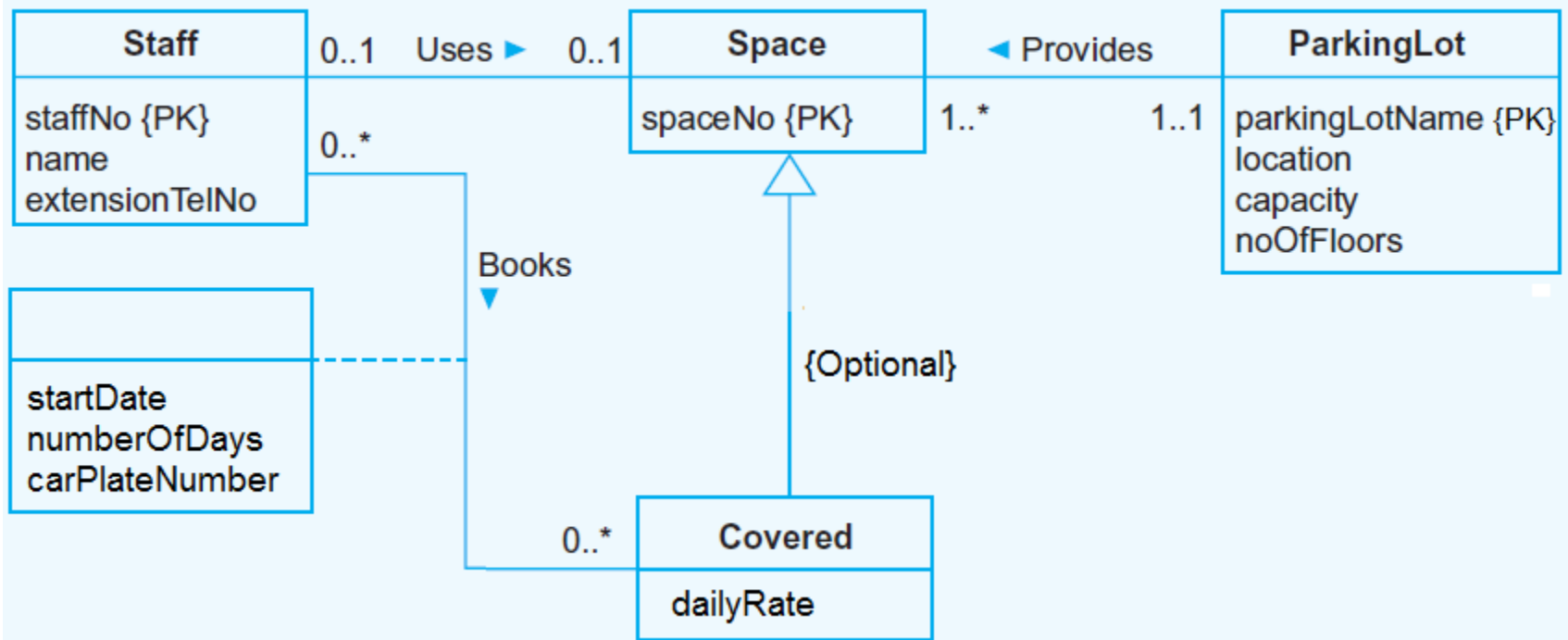


□ Introduce specialization/generalization concepts into the **ER** model to show the following:

- Some parking spaces are covered and each can be allocated for use by a member of staff or his/her visitors for a daily rate.
- To book a covered space, the staff member needs to provide the start date, the number of days, and the plate number of the car using the space.
- Parking spaces that are not covered (uncovered) are free to use and each can be allocated for use by a member of staff.

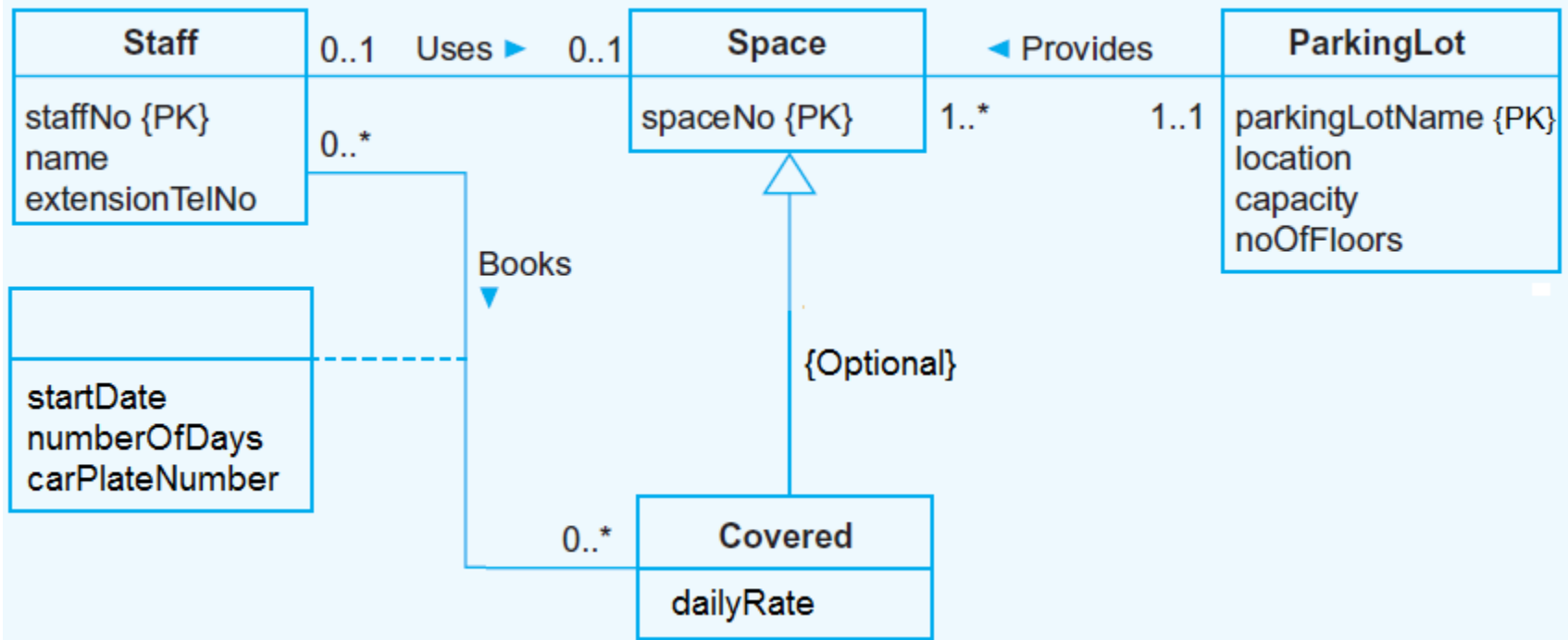
Solution

- To book a covered space, the staff member needs to provide the start date, the number of days, and the plate number of the car using the space.



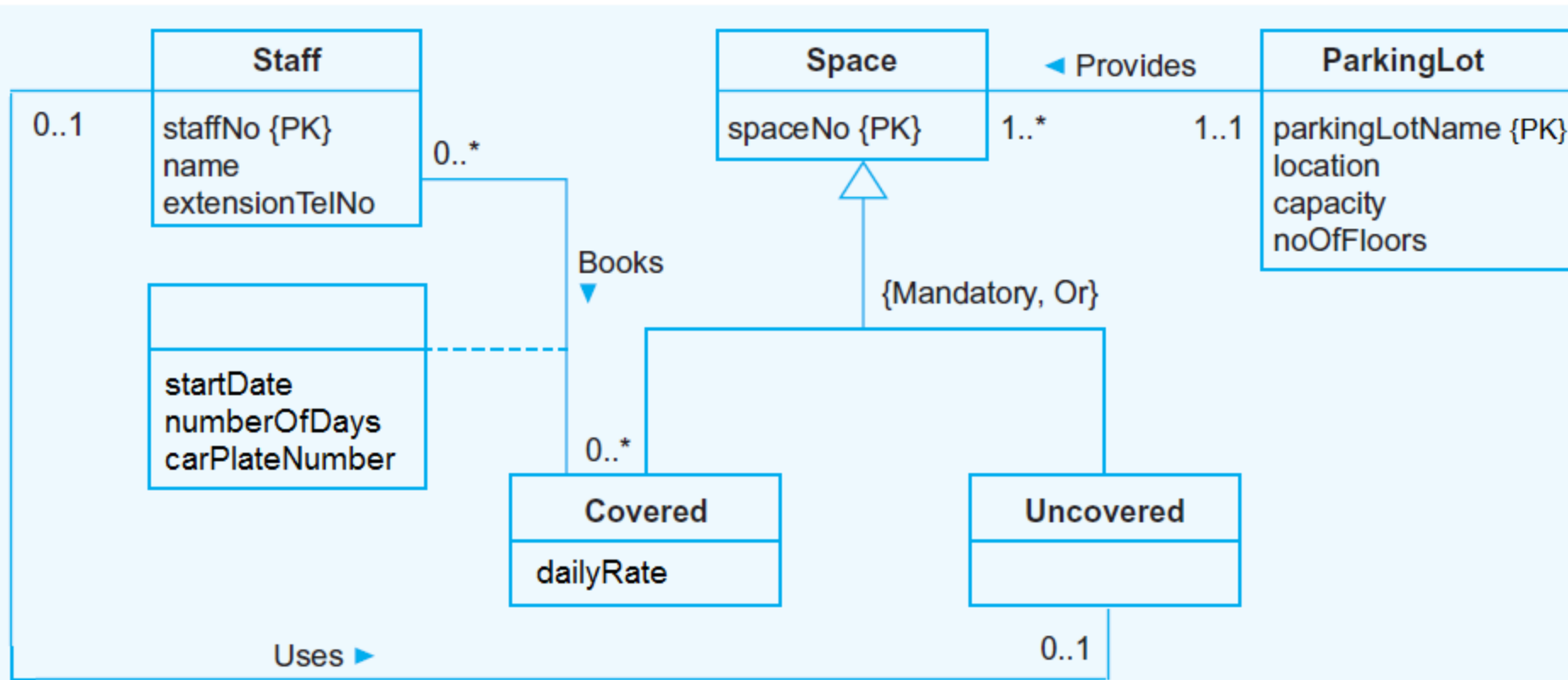
Exercise 1 - Update

- Covered parking spaces can be used **by visitors only**. Staff members use uncovered spaces.



Exercise 1 - Update

- Covered parking spaces can be used **by visitors only**. Staff members use uncovered spaces.

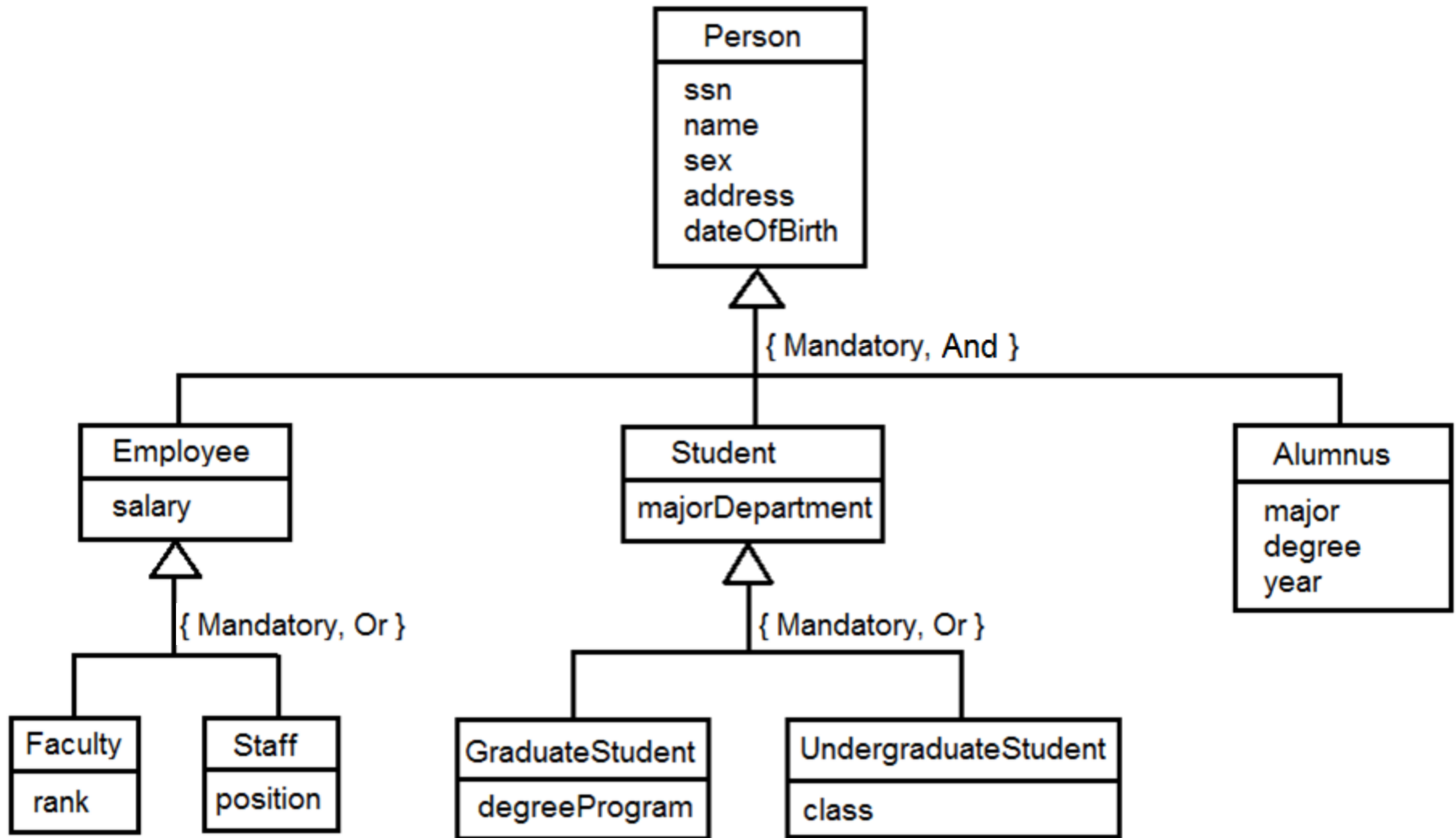


Exercise 2

- Apply the generalization process to the below entities and draw the *EER* model.

Faculty	Staff	GraduateStudent	UndergraduateStudent	Alumnus
ssn name sex address dateOfBirth salary rank	ssn name sex address dateOfBirth salary position	ssn name sex address dateOfBirth majorDepartment degreeProgram	ssn name sex address dateOfBirth majorDepartment class	ssn name sex address dateOfBirth major degree year

Solution



Exercise 3

- Apply the generalization process to the below entities and draw the *EER* model.

Faculty ssn name sex address dateOfBirth salary rank	Staff ssn name sex address dateOfBirth salary position	GraduateStudent ssn name sex address dateOfBirth majorDepartment degreeProgram	UndergraduateStudent ssn name sex address dateOfBirth majorDepartment class	Alumnus ssn name sex address dateOfBirth major degree year
ResearchAssistantStudent ssn name sex address dateOfBirth majorDepartment salary percentTime project	TeachingAssistantStudent ssn name sex address dateOfBirth majorDepartment salary percentTime course			

Solution

